

Punteros en C

Un **puntero** es una variable que guarda una **dirección de memoria**.

```
int numero = 10;  
int *p = &numero;
```

En este caso, *p* apunta a *numero*.

La diferencia entre las expresiones principales es:

numero → valor de la variable

&numero → dirección de memoria de *numero*

p → dirección guardada en el puntero

**p* → valor que hay en esa dirección

&p → dirección de memoria del propio puntero

Si imaginamos:

numero está en 1000

p está en 2000

entonces:

numero = 10

&numero = 1000

p = 1000

**p* = 10

&p = 2000

La idea más importante es:

p = ¿a dónde apunta el puntero?

**p* = ¿qué valor hay ahí?

&p = ¿dónde está guardado el puntero?

Además, **&** sirve para obtener una dirección de memoria y ***** sirve para acceder al contenido de la dirección apuntada.

Modificar una variable mediante un puntero

Un puntero también permite modificar el contenido de la variable apuntada.

```
int numero = 10;
int *p = &numero;
```

```
*p = 50;
```

Ahora:

```
numero = 50
```

Esto ocurre porque `*p` accede directamente al lugar donde está almacenado *numero*.

Punteros y funciones

Normalmente C pasa los parámetros **por valor**, es decir, entrega una copia:

```
void modificar(int numero) {
    numero = 50;
}
```

Esto no modifica la variable original. Con un puntero podemos enviar su dirección:

```
void modificar(int *p) {
    *p = 50;
}
```

Y llamar:

```
int numero = 10;

modificar(&numero);
```

Ahora sí *numero* termina valiendo 50, porque la función recibió su dirección y pudo acceder al dato original.

Relación con el scanf

Cuando hacemos:

```
int numero;
```

```
scanf("%d", &numero);
```

scanf() necesita saber **dónde guardar** el dato ingresado. Por eso no le damos solamente numero, sino: &numero. Es decir, su dirección de memoria.

Relación entre punteros y vectores

Los vectores están estrechamente relacionados con los punteros.

```
int numeros[3] = {10, 20, 30};
```

En la mayoría de las expresiones, numeros se convierte en un puntero al primer elemento. Por eso: *numeros representa numeros[0]. Y también existe la equivalencia:

```
numeros[0] == *(numeros + 0)
numeros[1] == *(numeros + 1)
numeros[2] == *(numeros + 2)
```

Esto también ayuda a entender por qué los vectores pueden modificarse dentro de una función.